

JSP Basics — Assignment Answers

Topics: static JSP, scriptlets, page directives, Expression Language (EL), JSP vs React

Q1. hello.jsp — Static 'Hello, JSP World!' Page

```
<html>
<head>
  <title>Hello JSP</title>
</head>
<body>
  <h1>Hello, JSP World!</h1>
</body>
</html>
```

Running it on Tomcat:

- 1 Place hello.jsp inside the WebContent/ (or src/main/webapp/) folder of the Dynamic Web Project.
- 2 Start the Tomcat server from Eclipse (right-click project → Run As → Run on Server).
- 3 Open a browser and visit <http://localhost:8080/MyWebApp/hello.jsp>.
- 4 The browser displays 'Hello, JSP World!' — Tomcat automatically compiled the .jsp file into a servlet behind the scenes before serving it.

Q2 & Q3. showUser.jsp — Dynamic Welcome Message + Date

Q2 uses a JSP scriptlet to read the 'username' request parameter. Q3 adds a page directive for UTF-8 encoding and imports java.time.LocalDate to show today's date.

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" language="java" %>
<%@ page import="java.time.LocalDate" %>
<html>
<head>
    <title>Show User</title>
</head>
<body>
    <%
        // Q2: read the 'username' parameter from the request URL
        String username = request.getParameter("username");
        if (username == null || username.trim().isEmpty()) {
            username = "Guest";
        }
    %>
    <h2>Welcome, <%= username %>!</h2>

    <%
        // Q3: display the current date using java.time.LocalDate
        LocalDate today = LocalDate.now();
    %>
    <p>Today's date: <%= today %></p>
</body>
</html>
```

Test URL: <http://localhost:8080/MyWebApp/showUser.jsp?username=Riya> → displays 'Welcome, Riya!' followed by today's date, e.g. 'Today's date: 2026-09-24'.

The two directives: the first `<%@ page ... %>` sets response content type/encoding to UTF-8 (so special characters render correctly); the second imports `LocalDate` so it can be used directly in the scriptlet without a fully-qualified name.

Q4. orderSummary.jsp — Expression Language (EL)

A small servlet sets the three request attributes and forwards to orderSummary.jsp, which reads them using EL (`${...}`) instead of scriptlets, and computes the total with EL arithmetic.

```
// TestOrderServlet.java - sets attributes then forwards
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    request.setAttribute("itemName", "Wireless Mouse");
    request.setAttribute("quantity", 3);
    request.setAttribute("price", 499.0);

    RequestDispatcher dispatcher = request.getRequestDispatcher("orderSummary.jsp");
    dispatcher.forward(request, response);
}
```

orderSummary.jsp:

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" language="java" %>
<html>
<head><title>Order Summary</title></head>
<body>
    <h2>Order Summary</h2>
    <p>Item: ${itemName}</p>
    <p>Quantity: ${quantity}</p>
    <p>Price per unit: ${price}</p>
    <p><b>Total Cost: ${quantity * price}</b></p>
</body>
</html>
```

Visiting TestOrderServlet forwards to orderSummary.jsp, which shows: Item: Wireless Mouse, Quantity: 3, Price per unit: 499.0, Total Cost: 1497.0 — all computed directly by EL, with no Java scriptlet code inside the JSP.

Q5. JSP vs React — Dynamic Rendering Comparison

JSP mixes Java code directly into HTML using scriptlets/EL and renders everything on the **server**, sending a finished HTML page to the browser for every request (e.g. `showUser.jsp` recompiles and re-renders the whole page each time the URL is hit). React instead renders in the **browser** using JavaScript components and JSX, updating only the parts of the page that change (via the virtual DOM) without a full page reload. In terms of data flow, JSP typically pulls data fresh from the server on each request (request parameters, session, or database), while React manages data as component state/props and re-renders reactively whenever that state changes, often fetching data asynchronously via APIs. JSP tends to suit simpler, page-per-request server-rendered apps (classic multi-page Java web apps), whereas React suits highly interactive single-page applications (SPAs) like modern dashboards or feeds that update live without reloading.